



Salesforce Developer Assessment

Customer Community Addendum

Experience Cloud · LWC · Cross-Surface Reuse

1. Why a Customer Community?

OpenGov sells to government agencies. Real customers — public-sector procurement officers — need self-service visibility into the hardware they purchased and the warranties protecting it. A Customer Community is the architectural layer that delivers that, without exposing internal CRM screens.

Equally important: building the Community proves the backend works end-to-end through an external user flow. Every Apex class, the trigger handler, the validations, the error cleaner — all get exercised by a real customer interaction. The Community is both a deliverable and the highest-fidelity integration test.

How Each Backend Component Is Exercised

Backend Component	Triggered By Customer Action
Record-Triggered Flow (Quote → Asset)	Sales rep contracts a Quote. Customer sees the new Asset on My Assets within seconds.
AssetService Apex (Asset → Task)	When Service team marks Asset Installed, Task appears on customer's My Service Requests.
WarrantyRegistrationTriggerHandler (addError)	Customer fills the registration form. Both cross-object and cross-record validations fire.
FlowErrorHandler Invocable cleaner	addError text is caught by the controller, cleaned by FlowErrorHandler, displayed in modal banner.
Warranty_Registration__c sharing model	Customer sees only their own account's warranties. ControlledByParent enforces this declaratively.

2. Community Architecture

2.1 Components

- customerDashboard — Three KPI tiles (Assets, Active Warranties, Open Tasks).
- customerAssets — Read-only datatable; columns include status badges and warranty indicator.
- customerWarranties — List + modal registration form. Showcase component for the addError flow.
- customerServiceRequests — Read-only Task list, populated automatically by AssetService.

2.2 Single Apex Controller

CommunityPortalController centralizes auth, scoping, and error handling. Every method:

- Resolves the user's AccountId via Contact link as the first action.
- Returns null/empty for unscoped users (defense in depth).
- Returns DTOs rather than raw SObjects (clean LWC contract, schema-independent).
- Reuses FlowErrorHandler in catch blocks (same cleaned errors across all surfaces).

3. Security Model

Five layers, each necessary; missing any one breaks the model:

Layer	Controls	Configured Where
License	Whether the user can see Salesforce data at all.	Customer Community Plus License
Profile / Permission Set	Object/field permissions (Read, Create, Edit).	Permission Set "Community Portal Access"
OWD	Default record-level access (typically private external).	Asset, Task, Warranty: ControlledByParent
Sharing Set	Grants community user access to records related to their Contact.Account.	Sharing Set in Community Workspaces
Apex (defensive)	Re-scopes every query to user's AccountId; uses WITH SECURITY_ENFORCED.	CommunityPortalController

4. The "Without Sharing" Decision for Tasks

Tasks have additional platform sharing layers in Communities, beyond standard objects. Even with the correct OWD, Sharing Set, and Permission Set, portal users sometimes cannot read Tasks because the "RelatedTo" polymorphic relationship triggers extra checks.

Rather than fight the platform, the solution uses Salesforce's documented Apex Sharing pattern: an inner class declared "without sharing" runs only the Task query. The outer class is "with sharing" and validates the user identity (Contact → Account). The inner class accepts only a pre-validated AccountId and returns Tasks for that specific Account — no parameter the user could manipulate.

The user's identity gates access; the helper executes the query the user is already entitled to make. Security boundary is preserved at the Apex authentication layer instead of at the platform sharing layer.